Inbox Pattern and Deduplication

receive at least once, process once

Phase 4 videos on YouTube

In one line: the inbox pattern stores each received message ID in the consumer's database in the same transaction as the work it triggers, so a redelivered message is detected and skipped; together with the outbox on the producer side, it gives effectively-once processing between services.

1 The same message, twice

The payment service receives `PaymentRequested` for order 42 and charges the customer's card. Then the same message is delivered again, maybe because the outbox relay re-published it, or the consumer crashed before committing its offset. Without protection, the customer is charged twice.



Figure 1. At-least-once systems create duplicates in many places. The consumer must handle all of them.

2 The inbox

The consumer keeps an `inbox` table (or processed_messages) with the **message ID as the primary key**. For each message, it opens a transaction, inserts the ID, does its work, and commits. If the ID is already there, the insert fails, the transaction rolls back, and the consumer simply acknowledges: the duplicate has no effect. Because the ID and the effects commit atomically, there's no window where one is saved without the other.

```
BEGIN;
INSERT INTO inbox (message_id, received_at) VALUES ('m-77', now());
--  duplicate key → ROLLBACK; just ack the message
INSERT INTO payments (order_id, amount, status) VALUES (42, 59.80, 'PENDING');
INSERT INTO outbox (...) VALUES (... 'PaymentRequested' ...);
COMMIT;
```

Figure 2. Inbox, business change and the next outbox event in one local transaction.

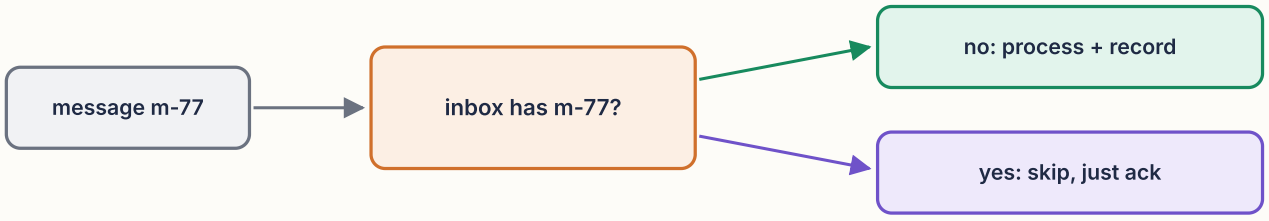


Figure 3. Check-and-record happens atomically through the unique key, not as a separate read.

WORKED EXAMPLE: IGNORING A REDELIVERED PAYMENTREQUESTED

The first delivery, with message ID `m-77`, opens a transaction, inserts `m-77` into the inbox, creates a PENDING payment and writes a `ChargeCard` outbox event, then commits and acks. The card is charged with idempotency key `m-77`. Ten minutes later the relay re-publishes the same event after a restart. The inbox insert fails with a duplicate key, the consumer rolls back and acks, and nothing else happens. The inbox row expires after 14 days, longer than any possible redelivery.

3 Practical details

- Use the **producer's event ID**, stable across retries, not a broker delivery tag.
- Keep inbox rows longer than the redelivery and replay window; clean up with a TTL job or partitions.
- Side effects outside your database (card APIs, email) still need their own idempotency keys.
- Where natural idempotency exists (an upsert by order ID), the inbox may be unnecessary.

COMMON MISTAKES

- SELECT to check the inbox, then INSERT later in another transaction (race condition).
- Deduplicating in memory or in Redis without a transaction with the effect.
- Inbox retention shorter than the topic's replay window.

4 Key terms

Inbox pattern: record received message IDs with the work, atomically

Message ID: stable unique ID assigned by the producer

Deduplication window: how long IDs are remembered

Effectively-once: at-least-once delivery + dedup = one effect

5 Interview questions

Q1. What is the inbox pattern? BASIC

Storing each processed message ID in the consumer's database, in the same transaction as the work, so duplicates are detected by the unique key and skipped.

Q2. How do outbox and inbox work together? BASIC

The outbox guarantees events are published at least once; the inbox makes processing them idempotent, so each event's effect happens once.

Q3. Customers are occasionally charged twice. Messages come from Kafka. Fix it. SCENARIO

Add an inbox keyed by event ID in the same transaction as recording the payment, and pass the event ID as the idempotency key to the card provider.

Q4. How long do you keep inbox rows? SCENARIO

Longer than any duplicate can arrive: broker retention, retry and DLQ replay windows; then delete with a scheduled job.

Q5. Why not check 'already processed?' with a SELECT first? TRICKY

Two deliveries processed concurrently can both see 'not processed' and both run. The unique insert in the same transaction makes the check atomic.

Q6. Does an inbox protect external API calls? TRICKY

Not by itself; the external call isn't in your transaction. Use the provider's idempotency key, or record intent first and make the call idempotent.

Next: SD-I-017 · Schema Evolution and Registries